# *Interface*

- Why we need Interface?
- What is Interface?
  - *Behaviour definition*
  - *No implements keyword*
- Empty Interface
- Interface Embedding

# *Interface*

- There is no explicit declaration of intent, no "implements" keyword.
- interface values can be thought of as a tuple of a value and a concrete type:
  - *(value, type)*

# *Interface*

Output?

```go
package main

import "fmt"

type MyError struct{}

func (e *MyError) Error() string {
    return "something went wrong"
}

func getError() error {

    var err *MyError = nil

    return err
}

func main() {

    err := getError()

    if err == nil {

        fmt.Println("No error")

    } else {

        fmt.Println("Error exists")
    }
}
```

# *Interface and Polymorphism*

```go
type Payment interface{
    Pay(amount float64)
}

type UPI struct{
    Id string
}

type CreditCard struct{
    Number string
}
```

```go
func (u UPI) Pay(amount float64) {
    fmt.Println("Paid using UPI:", amount)
}

func (c CreditCard) Pay(amount float64) {
    fmt.Println("Paid using Credit Card:", amount)
}

func MakePayment(p Payment, amount float64){
    p.Pay(amount)
    // cannot access struct fields
}
```

# *Interface: Remember these*

- Interface internally stores (Type, Value)
- Interface is nil only when BOTH Type and Value are nil
- Interface holding nil pointer is NOT nil
- Interface enables polymorphism
- Interface implementation is implicit (no implements keyword)

# *Interface: Practice*

```go
var i interface{}
```

vs

```go
var p *User = nil
var i interface{} = p
```

# *Interface: Practice Questions*

Q1. What is the internal representation of an interface in Go?

Q2. What is a nil interface? How is it different from an interface holding a nil pointer?

Q3. Why is an interface holding a nil pointer NOT equal to nil?

Q4. What are the two components stored inside an interface?

Q5. What is an empty interface?

# *Interface: Practice Questions*

Q6. When should you use pointer receiver vs value receiver for interface implementation?

Q7. Can a pointer type implement an interface but value type cannot? Why?

Q8. Can an interface implement another interface?

Q9. Why are interfaces called implicit in Go?

Q10. What is method set?

# *Type assertion*

Type assertion extracts concrete value.

```
var i interface{} = 10

value := i.(int)
```

# *Type Switch*

Switch on type.

```go
switch v := i.(type) {
case int:
    fmt.Println("int", v)
case string:
    fmt.Println("string", v)
default:
    fmt.Println("unknown")
}
```

# *Functions*

# *Functions: defer*

- defer is used to delay the execution of a function until the surrounding function finishes.
- It is heavily used in resource cleanup like closing files, DB connections, unlocking mutex,
- LIFO = Last In First Out order
- Arguments are evaluated immediately
- Defer runs when function returns

# *Interview Questions*

```go
package main
import "fmt"

func main() {

    defer fmt.Println("1")

    defer fmt.Println("2")

    defer fmt.Println("3")

}
```

# *Defer: Arguments are evaluated immediately*

```go
package main
import "fmt"

func main() {

    x := 10

    defer fmt.Println(x)

    x = 20
}
```

Output: 10

# *Interview Question*

```go
func main() {

    for i := 1; i <= 3; i++ {

        defer fmt.Println(i)
    }
}
```

# *Defer: Things to Remember*

- defer runs after function finishes
- multiple defer run in reverse order
- arguments evaluated immediately
- used for cleanup
- follows stack (LIFO)